



МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования

«МИРЭА – Российский технологический университет»

РТУ МИРЭА

Практическая работа № 3/4 ч.

Разработка мобильных приложений

Уровень	(наименование дисциплины (модуля) в соответствии с учебным планом) бакалавриат
Форма обучения	(бакалавриат, магистратура, специалитет) очная
Направление(-я) подготовки	(очная, очно-заочная, заочная) 09.03.02 «Информационные системы и технологии»
Институт	(код(-ы) и наименование(-я)) кибербезопасности и цифровых технологий
Кафедра	(полное и краткое наименование) КБ-14 «Цифровые технологии обработки данных»
Используются в данной редакции с учебного года	(полное и краткое наименование кафедры, реализующей дисциплину (модуль)) 2024/25
Проверено и согласовано « ____ » ____ 20 ____ г.	(учебный год цифрами) (подпись директора Института/Филиала с расшифровкой)

Москва 2024 г.

ОГЛАВЛЕНИЕ

ВВЕДЕНИЕ.....	3
1.1 MVC (Model-View-Controller).....	3
1.2 MVP (Model-View-Presenter).....	4
1.3 MVVM (Model-View-ViewModel)	4
1.4 MVI (Model-View-Intent)	5
1.5 VIPER (View-Interactor-Presenter-Entity-Router)	6
1.6 Сравнение паттернов.....	7
2 МОДИФИКАЦИЯ СЛОЯ APP	7
3 КОНТРОЛЬНОЕ ЗАДАНИЕ.....	15

ВВЕДЕНИЕ

Архитектурные шаблоны MVC, MVP, MVVM, MVI, и VIPER используются для структурирования и разделения ответственности в разработке приложений с пользовательским интерфейсом (UI). Каждый из них решает задачу отделения логики интерфейса от бизнес-логики и данных, но с разными акцентами и подходами. Рассмотрим каждый из них подробно.

1.1 MVC (Model-View-Controller)

MVC (Модель-Представление-Контроллер) – один из самых популярных и простых архитектурных шаблонов, особенно в веб-разработке.

Компоненты:

- **Model (Модель)**: содержит данные и бизнес-логику. Модель не зависит от представления и отвечает за хранение и обработку данных.
- **View (Представление)**: отвечает за отображение данных пользователю и приём взаимодействий (например, нажатия кнопок).
- **Controller (Контроллер)**: управляет взаимодействием между View и Model. Контроллер принимает действия от пользователя, передает их модели, а затем обновляет представление.

Принцип работы:

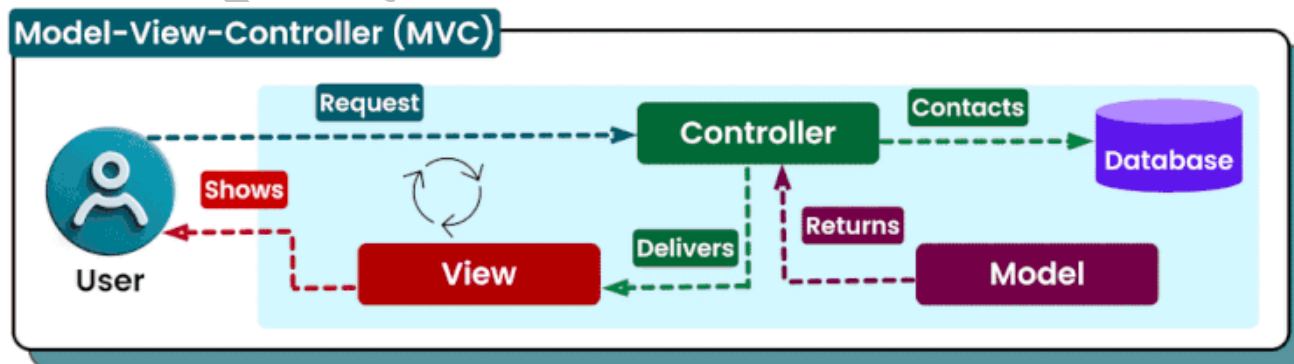
1. **View** отображает интерфейс и взаимодействует с пользователем.
2. **Controller** получает действия пользователя, обновляет **Model**.
3. **Model** обновляет данные, после чего **Controller** обновляет **View**.

Преимущества:

- Простая архитектура для небольших и средних приложений.
- Чёткое разделение обязанностей.

Недостатки:

- В больших приложениях контроллер может стать перегруженным.
- Контроллер напрямую управляет обновлением View, что усложняет поддержку.



1.2 MVP (Model-View-Presenter)

MVP (Модель-Представление-Презентер) — это эволюция MVC, в которой больше внимания уделяется отделению логики View от логики управления.

Компоненты:

- **Model (Модель)**: обрабатывает данные и бизнес-логику.
- **View (Представление)**: отображает данные и взаимодействует с пользователем, но вся логика делегируется презентеру.
- **Presenter (Презентер)**: управляет взаимодействием между View и Model. Презентер принимает действия от View, изменяет Model и вручную обновляет View.

Принцип работы:

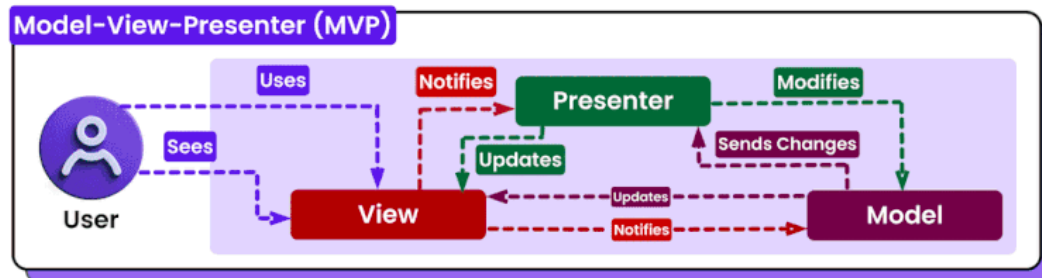
1. **View** передаёт взаимодействия пользователя в **Presenter**.
2. **Presenter** обновляет **Model** и получает обновленные данные.
3. **Presenter** вручную обновляет **View**.

Преимущества:

- Легко тестировать, так как **Presenter** отделен от **View**.
- Контроль над обновлением UI.

Недостатки:

Презентер может стать слишком сложным при большом количестве логики.



1.3 MVVM (Model-View-ViewModel)

MVVM (Модель-Представление-Модель Представления) — это архитектурный шаблон, который активно используется в средах с привязкой данных, таких как WPF, Xamarin и SwiftUI.

Компоненты:

- **Model (Модель)**: как и в других паттернах, отвечает за бизнес-логику и данные.
- **View (Представление)**: Пользовательский интерфейс, который взаимодействует с ViewModel через привязку данных.

- **ViewModel (Модель представления)**: управляет логикой отображения и данными для View. Связывается с View через двустороннюю привязку данных.

Принцип работы:

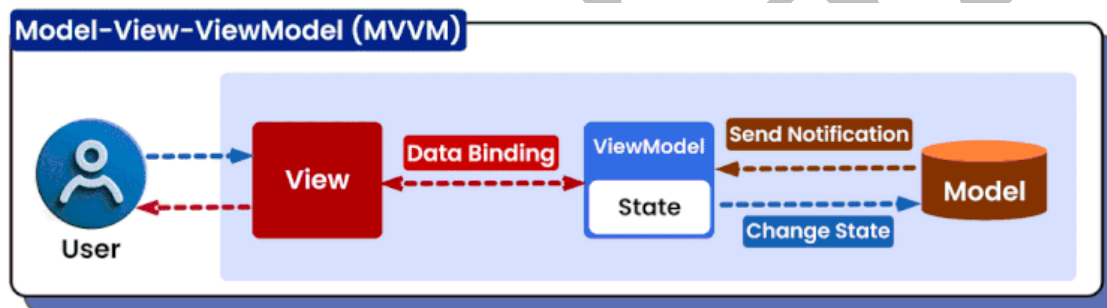
1. **View** привязана к **ViewModel** через data binding.
2. **ViewModel** изменяет данные в **Model**, и обновления автоматически отображаются в **View**.

Преимущества:

- удобно для сложных интерфейсов с большим количеством данных, так как автоматическая привязка упрощает работу с UI.
- хорошо тестируется, так как **ViewModel** отделен от **View**.

Недостатки:

- Может быть сложно реализовать, если нет встроенной поддержки привязки данных.
- Более сложный по сравнению с MVC и MVP.



1.4 MVI (Model-View-Intent)

MVI (Модель-Представление-Интент) — это реактивная архитектура, ориентированная на однонаправленный поток данных. Она особенно популярна в Android-разработке с использованием RxJava или Kotlin Coroutines.

Компоненты:

- **Model (Модель)**: обрабатывает данные и бизнес-логику.
- **View (Представление)**: отображает состояние приложения и принимает пользовательский ввод.
- **Intent (Интент)**: это действия или события, которые инициируют изменения состояния.

Принцип работы:

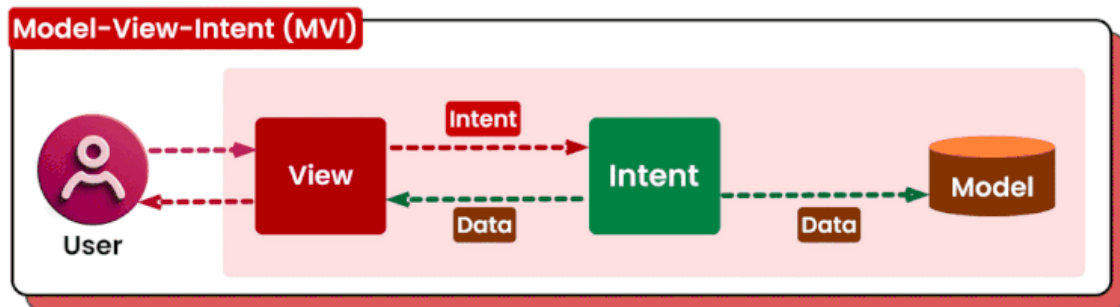
1. **View** передаёт действия пользователя как **Intent**.
2. **Intent** вызывает изменения в **Model**, которые порождают новое состояние.
3. **Model** отправляет обновленное состояние обратно в **View**.

Преимущества:

- чёткий однонаправленный поток данных, который улучшает предсказуемость поведения.
- легко тестировать и отлаживать.

Недостатки:

требует работы с состояниями и более сложного управления потоками данных, что может увеличивать сложность приложения.



1.5 VIPER (View-Interactor-Presenter-Entity-Router)

VIPER — это модульный архитектурный шаблон, который активно используется в разработке iOS-приложений. Он предлагает очень чёткое разделение обязанностей между компонентами.

Компоненты:

- **View (Представление)**: отображает данные и передает события пользователю.
- **Interactor (Интерактор)**: управляет бизнес-логикой и запросами к данным.
- **Presenter (Презентер)**: получает события от View и передает их в Interactor. Обновляет View на основе ответа от Interactor.
- **Entity (Сущность)**: представляет бизнес-объекты.
- **Router (Роутер)**: управляет навигацией между экранами.

Принцип работы:

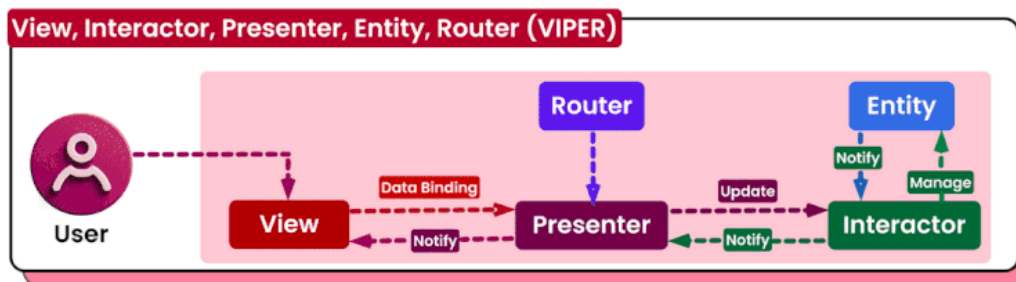
1. **View** передает события пользователю в **Presenter**.
2. **Presenter** передает запросы в **Interactor** для обработки бизнес-логики.
3. **Interactor** обрабатывает данные, взаимодействует с **Entity** и возвращает результат в **Presenter**.
4. **Presenter** обновляет **View** и, при необходимости, инициирует навигацию через **Router**.

Преимущества:

- очень чёткое разделение обязанностей, что повышает тестируемость и поддерживаемость.
- хорошо структурирована для крупных проектов.

Недостатки:

- много компонентов для каждого модуля, что может усложнять проект.
- трудоемкость в реализации для простых приложений.



1.6 Сравнение паттернов

Шаблон	Основные компоненты	Когда использовать
MVC	Model, View, Controller	Для простых и средних приложений, часто в веб-разработке.
MVP	Model, View, Presenter	Для приложений с ручным управлением UI, мобильные приложения (например, Android).
MVVM	Model, View, ViewModel	Для приложений с привязкой данных (WPF, Xamarin, SwiftUI, Android = databinding).
MVI	Model, View, Intent	Для реактивных приложений, где важен однонаправленный поток данных.
VIPER	View, Interactor, Presenter, Entity, Router	Для крупных проектов с iOS, где важно модульное разделение обязанностей.

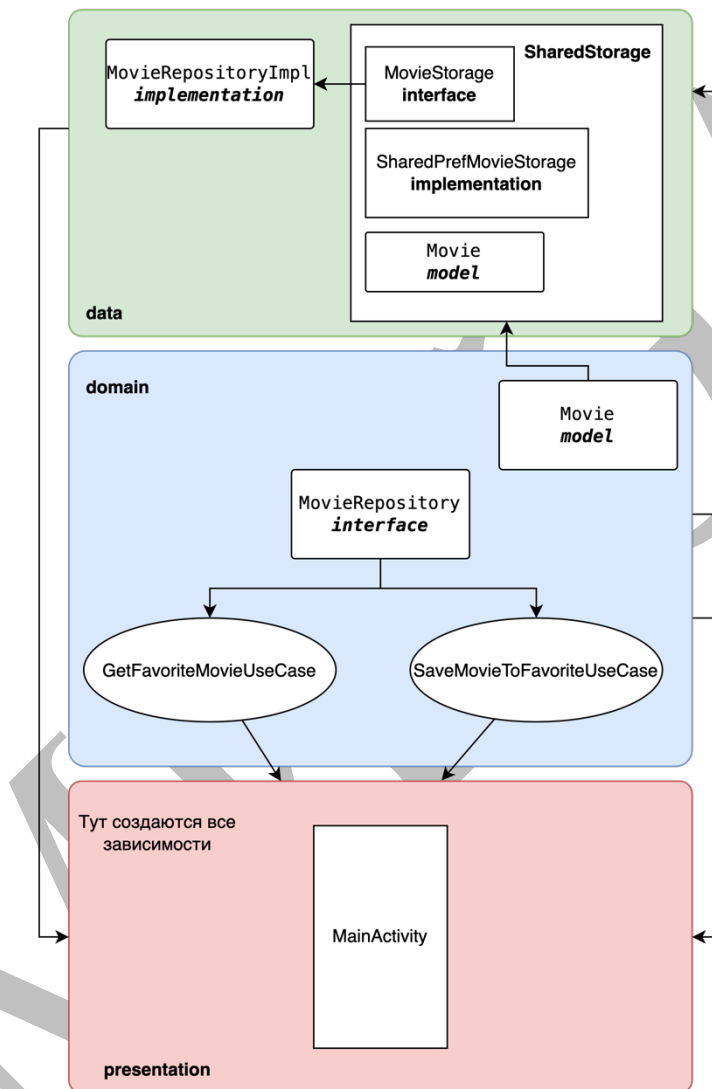
Таким образом,

- **MVC** — лучший выбор для небольших веб-приложений с простой логикой.
- **MVP** — полезен для мобильных приложений с большим количеством пользовательских взаимодействий.
- **MVVM** — хорошо подходит для приложений, где используется data binding для автоматического обновления интерфейса.
- **MVI** — эффективен для реактивных приложений с однонаправленным потоком данных.
- **VIPER** — отличная архитектура для крупных приложений, требующих четкой модульной структуры (например, в iOS-разработке).

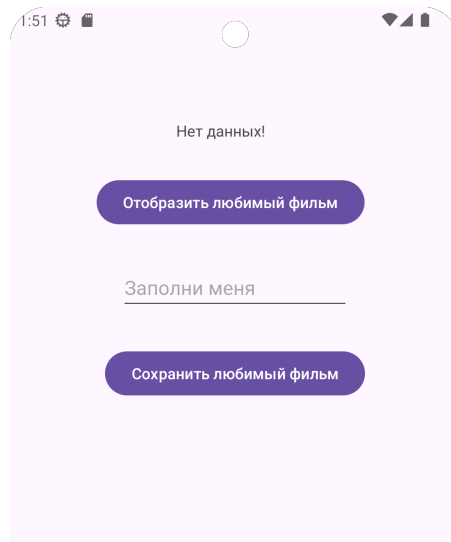
2 МОДИФИКАЦИЯ СЛОЯ APP

Задание. Требуется открыть приложение из практического занятия №1. Ваша реализация и смысл приложения отличаются от представленного в практическом задании примера. Необходимо изменить приложение по аналогии с примером.

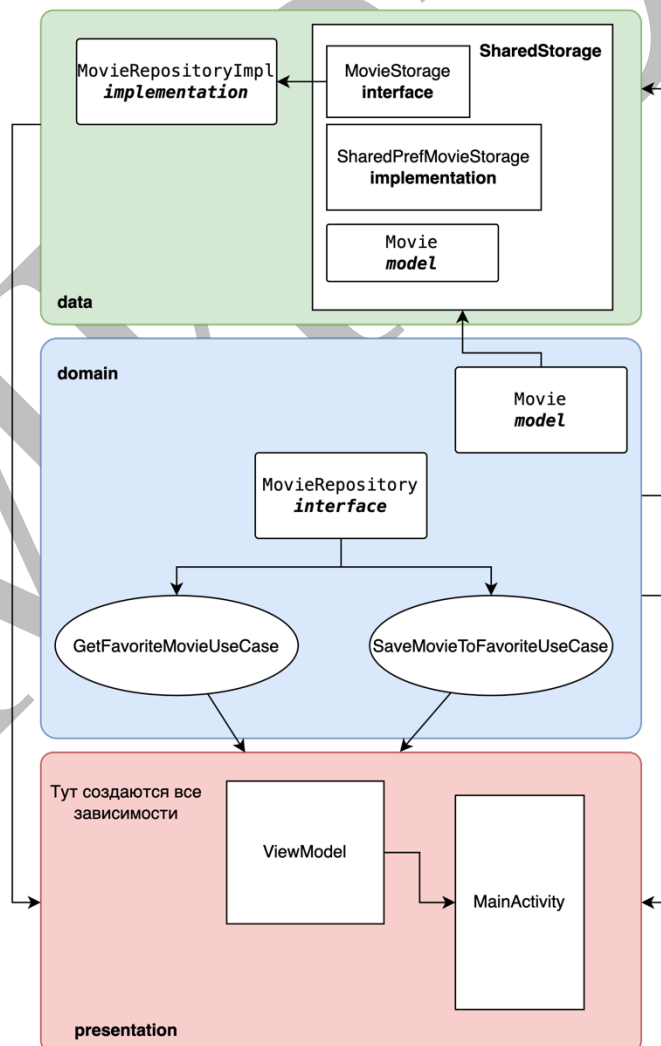
Возвращаемся к теме «Чистая архитектура». На рисунке представлена структура проекта, который была реализована на предыдущем занятии. Слой презентейшн содержит Activity. Activity общается с двумя юзкейсами. Юзкейсы в свою очередь имеют доступ к Movie-репозиторию. В слое domain содержится одна модель. В слое *data* представлена реализация *MovieRepositoryImplementation*. Хранение данных осуществляется с помощью механизма *SharedPreferences*. В рамках проекта произведено разделение на три модуля.



На текущем занятии рассмотрен модуль App, но в части Presentation. На примере MVVM будет реализована интеграция архитектурного шаблона в проект. Класс *MainActivity* будет декомпозирован на две составляющих части. Одна часть предназначена для работы с интерфейсом, то есть *TextView*, *EditText* и кнопки управления, а вторая часть — это логическая часть, которая отвечает за взаимодействие с use cases — загружает данные и в более удобном виде возвращает данные Activity.



Далее следует модифицировать диаграмму. В слое app необходимо добавить дополнительно еще один прямоугольник, который является представлением view-модели (что такое ViewModel – было рассмотрено в рамках прошлого семестра). Все use-кейсы теперь взаимодействуют только с view-моделью, а activity будет слушать view-модель.



Зачем это нужно делать? Во-первых, будет перенесена часть логики из Activity во ViewModel, т.е. в каждом из этих классов будет еще меньше логики. Вторая причина заключается в том, что Activity не решает полноценно вопрос сохранения состояния. Метод onCreate внутри Activity вызывается каждый раз, когда происходит поворот экрана. Т.е. будут выполняться все действия, вся загрузка, все обращение к use-case повторно. ViewModel как раз предназначена для решения этой проблемы за счет того, что имеет устойчивый жизненный цикл. После изменения Activity происходит считывание последнего состояния из ViewModel переменных, то есть ViewModel не будет обращаться повторно к UseCase и загружать какие-либо данные. Поэтому ViewModel является хранилищем для состояний Activity и знает что на экране отображается.

Особенность View-модели заключается в том, что View-модель ничего не знает об Activity несмотря на то, что стрелочка направлена навстречу. View-модель находится внутри Activity, то есть Activity знает о View-модели, но View-модель ничего не знает об Activity

Необходимо создать класс, наследуемый от базового класса ViewModel. Название класса возможно указать по примеру названия активности. Так как Activity называется MainActivity, значит и ViewModel возможно назвать MainViewModel. Импортирование родительского класса необходимо осуществить из пакета *android.lifecycle*.

```
public class MainViewModel extends ViewModel {  
}
```

Компонент ViewModel — предназначен для хранения и управления данными, связанными со слоем представления, а заодно решает проблемы с пересозданием активности. Стоит отметить, что не стоит воспринимать ViewModel заменой *onSaveInstanceState*, поскольку, после того как операционная система уничтожает активности (например, когда пользователь переходит в другое приложение) ViewModel будет также уничтожена и не сохранит своего состояния. В целом же, компонент ViewModel возможно охарактеризовать как синглтон с коллекцией экземпляров классов ViewModel, гарантирующий жизненный цикл пока имеется активный экземпляр активности. Стоит также отметить, что возможно выполнить привязку любого количества ViewModel к Activity(Fragment).

Класс ViewModel, сам по себе представляет абстрактный класс, без абстрактных методов и с одним protected методом *onCleared()*. Если необходимо очистить ресурсы, то возможно переопределить метод *onCleared()*, который вызывается когда связанная с ViewModel активность будет уничтожена. Стоит отметить, что во избежание утечки памяти, не нужно ссылаться напрямую на View или Context Activity из ViewModel. В целом, ViewModel должна быть абсолютно изолированная от представления данных.

```
public class MainViewModel extends ViewModel {

    public MainViewModel(
        ) { /* Init ViewModel here */
        Log.d(MainViewModel.class.getSimpleName().toString(), "MainViewModel
created");
    }

    @Override
    protected void onCleared() {
        Log.d(MainViewModel.class.getSimpleName().toString(), " MainViewModel
cleared");
        super.onCleared();
    }
}
```

Далее перейдем в MainActivity. В Activity в методе OnCreated необходимо добавить метод логирования и создать ViewModel. В первую очередь создадим поле класса vm. Инициализацию переменной необходимо проводить особенным образом. Но прежде, возможно проверить работу ViewModel при инициализации обычным способом. Посмотрим, как это все работает, а потом создадим правильным способом и сравним жизненный цикл.

```
public class MainActivity extends AppCompatActivity {

    private MainViewModel vm;
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        ...

        Log.d(MainActivity.class.getSimpleName().toString(), "MainActivity created");
        vm = new MainViewModel();
    }
}
```

Далее выполним запуск проекта и перейдем в *logcat*. Активити отображается на экране.

Задание: что отобразится в logcat при повороте экрана?

Далее необходимо верное связывание ViewModel и активности, чтобы ViewModel не пересоздавалась.

Правильная инициализация ViewModel осуществляется с помощью метода get класса ViewModelProvider.

```
vm = new ViewModelProvider(this).get(MainViewModel.class);
```

Класс ViewModel требует в качестве входного аргумента объект класса *ViewModelStoreOwner*. На данном шаге возможна ошибка. Необходимо проверить родительский класс MainActivity. Активность должна наследоваться не от Activity, а от *AppCompatActivity*. Далее необходимо запустить проект.

Произошло создание Activity и создалась ViewModel. Произведем поворот экрана устройства. Т.е., производятся повороты экрана, но ViewModel не

пересоздается.

```
com.mirea.fio.lecture2      D MainViewModel onCreate
com.mirea.fio.lecture2      D MainActivity onCreate
com.mirea.fio.lecture2      I Skipped 32 frames! The application may be doing too much work on its main thread.
com.mirea.fio.lecture2      D MainActivity onCreate
com.mirea.fio.lecture2      D MainActivity onCreate
```

При закрытии приложения произойдет вызов метода *onCleared*. При повторном открытии приложения произойдет повторное создание Activity и ViewModel. Т.е. если ViewModel создана, то возвращается ссылка на тот же самый объект. Т.е. ViewModel является Singleton в рамках жизненного цикла Activity.

Метод *onCleared* позволяет высвободить занимаемые ресурсы.

Следующий этап заключается в переносе логики работы с usecase во ViewModel. Стоит обратить внимание на тот факт, что ViewModel необходимо избавлять от зависимостей Android SDK, т.е. данный класс не должен зависеть от UI-компонентов, контекста и т.д. *Однако, каким образом решить вопрос создания экземпляра класса MovieStorage, требующий контекст?*

```
MovieStorage sharedPrefMovieStorage = new SharedPrefMovieStorage(this);
MovieRepository movieRepository = new
MovieRepositoryImpl(sharedPrefMovieStorage);
```

В первую очередь необходимо убрать логику создания реализации MovieRepository из ViewModel:

```
public class MainViewModel extends ViewModel {
    private MovieRepository movieRepository;

    public MainViewModel(MovieRepository movieRepository) {
        Log.d(MainViewModel.class.getSimpleName().toString(), "MainViewModel
created");
        this.movieRepository = movieRepository;
    }

    @Override
    protected void onCleared() {
        Log.d(MainViewModel.class.getSimpleName().toString(), "MainViewModel
cleared");
        super.onCleared();
    }

    public boolean setText(Movie movie) {
        Boolean result = new
SaveMovieToFavoriteUseCase(movieRepository).execute(movie);
        return result;
    }

    public Movie getText() {
        Movie movie = new GetFavoriteMovieUseCase(movieRepository).execute();
        return movie;
    }
}
```

Далее возможно создать класс, который отвечает за создание *ViewModelFactory* и в нем переопределить метод *create*:

```

public class ViewModelFactory implements ViewModelProvider.Factory {
    private Context context;

    public ViewModelFactory(Context context) {
        this.context = context;
    }

    @NonNull
    @Override
    public <T extends ViewModel> T create(@NonNull Class<T> modelClass) {
        MovieStorage sharedPrefMovieStorage = new
        SharedPrefMovieStorage(context);
        MovieRepository movieRepository = new
        MovieRepositoryImpl(sharedPrefMovieStorage);
        return (T) new MainViewModel(movieRepository);
    }
}

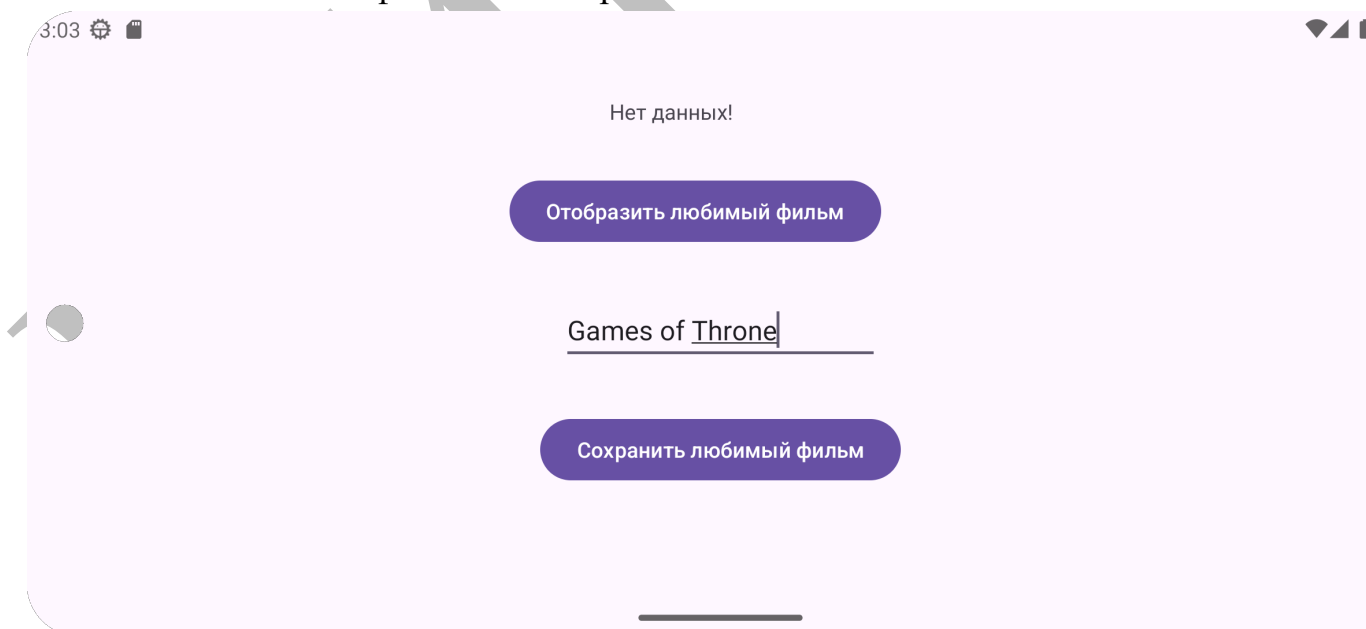
```

Метод *create* отвечает за создание *ViewModel* и её зависимостей, что позволяет изолировать *ViewModel* от всех внешних компонентов. Используя фабрику для создания *ViewModel*, решился вопрос зоны ответственности создания зависимостей, что существенно снизило связность кода.

Создание *ViewModel* с использованием фабрики предполагает передачу экземпляра класса вторым аргументом во *ViewModelProvider*.

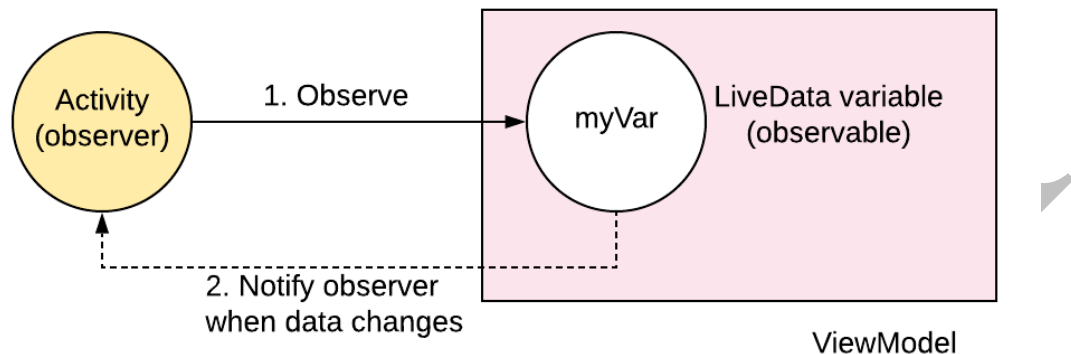
```
vm = new ViewModelProvider(this, new ViewModelFactory(this)).get(MainViewModel.class);
```

Далее необходимо проверить функционирование программы. Вся логика должна выполняться. Однако, что будет, в случае поворота экрана. При повороте экрана значение любимого фильма не сохранится.



Одним из способов решения этой проблемы будет являться сохранение данных в поле класса. Однако данный вариант будет неверным.

В таком случае появляется вопрос: А каким же образом возможно уведомить представление (Activity/Fragment) об изменениях в наших данных? В этом случае на помощь приходит LiveData. LiveData является классом, который хранит объект данных и позволяет наблюдателям получать обновления при каждом изменении данных. Основана на реализации паттерна «Observer» (наблюдатель). LiveData учитывает жизненный цикл, то есть обновляет наблюдателей, только когда они находятся в активном состоянии.



Ключевыми особенностями LiveData являются:

- **учет состояния жизненного цикла.** LiveData ориентирован на жизненный цикл, что означает автоматическое управление подписчиками на основе состояния жизненного цикла компонентов. Это предотвращает потенциальные утечки памяти и гарантирует выполнение обновлений пользовательского интерфейса, только когда компонент активен.
- **автоматические обновления.** LiveData уведомляет наблюдателей, когда хранящиеся в нем данные изменяются. Это устраняет необходимость в ручной обработке обновлений пользовательского интерфейса и сокращает объем кода.
- **согласованность данных.** LiveData гарантирует получение наблюдателями самых свежих данных. Когда состояние жизненного цикла изменяется (например, при переходе с фонового режима на передний план), LiveData запускает обновление, предоставляя наблюдателям актуальные данные.

В первую очередь в классе MainViewModel необходимо создать переменную типа MutableLiveData и метод, возвращающий значение:

```
private MutableLiveData<String> favoriteMovie = new MutableLiveData<>();

public MutableLiveData<String> getFavoriteMovie() {
    return favoriteMovie;
}
```

Так как LiveData является реализацией паттерна observer и все кто подписаны на изменение значений переменной получают уведомление, то методы getText и setText не должны возвращать значения, а просто обновлять их у переменной favoriteMovie.

```

public void setText(Movie movie){
    Boolean result = new
    SaveMovieToFavoriteUseCase(movieRepository).execute(movie);
    favoriteMovie.setValue(result.toString());
}

public void getText(){
    Movie movie = new GetFavoriteMovieUseCase(movieRepository).execute();
    favoriteMovie.setValue(String.format("My favorite movie is %s",
    movie.getName()));
}

```

Далее, в MainActivity необходимо подписаться на изменения переменной и обеспечить установку значений для textView

```

mainViewModel.getFavoriteMovie().observe(this, new Observer<String>() {
    @Override
    public void onChanged(String s) {
        textView.setText(s);
    }
});

findViewById(R.id.buttonSaveMovie).setOnClickListener(new
View.OnClickListener() {
    @Override
    public void onClick(View view) {
        mainViewModel.setText(new Movie(2, text.getText().toString()));
    }
});

findViewById(R.id.buttonGetMovie).setOnClickListener(new
View.OnClickListener() {
    @Override
    public void onClick(View view) {
        mainViewModel.getText();
    }
});

```

Стоит обратить внимание, что после поворота экрана LiveData оповестит активности о последнем установленном значении для переменной favoriteMovie.

3 КОНТРОЛЬНОЕ ЗАДАНИЕ

В ходе выполнения практического задания № 1 были сформулированы use-case возможности проекта. Приложение должно удовлетворять следующим функциональным требованиям:

1. Авторизация в приложении.
2. Взаимодействия с каким-либо внешним сервисом, возвращающим данные о стоимости валюты, погоды и т.д. в виде json-формата. Возможно использовать сервисы типа <https://www.wiremock.io/>, <https://mockapi.io/>, <https://github.com/typicode/json-server>, <https://thecatapi.com/> и т.д.
3. Сохранение данных в БД.
4. Отображение списка каких-либо сущностей с изображениями.
5. Отображение страницы сущности.

6. Различные возможности у авторизованного и гостевого пользователя.
7. Использование обученной модели для распознавания изображений, аудио, маски и т.д. (нужно произвести анализ существующих обученных моделей TensorFlow Lite, которые желаете использовать в приложении).

Задание.

1. Активности взаимодействие со слоем domain должно осуществлять через ViewModel.
2. Обновление состояния интерфейса осуществлять через LiveData.
3. Самостоятельно изучить MediatorLiveData и получить данные из замканных данных из сетевой части и БД.